DAM

PROMETEO by thePower

Tema 2: **Introducción a Dart**

Desarrollo de Interfaces· 2º DAM

Olga M. Moreno Martín

Septiembre 2026

PROMETEO
by thePower



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons
Olga M. Moreno Martín

https://creativecommons.org/licenses/by-nc-nd/4.0/

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Índice

01 **Introducción a Dart**
Rendimiento, ecosistema y simplicidad

02 **Configuración y Herramientas**
SDK, IDEs y primer proyecto

03 **Sintaxis Básica**
Variables, tipos y control de flujo

04 **Funciones Avanzadas**
Parámetros, closures y arrow functions

05 **Colecciones y Null Safety**
Listas, Sets, Maps y seguridad de nulos

06 **POO en Dart**
Clases, herencia, mixins e interfaces

07 **Características Avanzadas**
Extensions, generators y callable classes

08 **Asincronía y Concurrencia**
Futures, Streams e Isolates

09 **Buenas Prácticas**
Convenciones, código limpio y organización

10 **Documentación y Pruebas**
Testing unitario y cobertura de código

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

¿Por qué Dart?

Rendimiento

JIT (Just-In-Time): Compila durante la ejecución, ideal para desarrollo con Hot Reload en Flutter.

AOT (Ahead-Of-Time): Compila a código máquina nativo antes de ejecutarse, optimizando el rendimiento en producción.

Ecosistema Flutter

Dart es el lenguaje oficial de Flutter, el framework de Google para crear aplicaciones móviles, web y de escritorio con un solo código.

UI consistente | Comunidad activa

Simplicidad

Sintaxis clara y moderna con null safety integrada. Combina programación orientada a objetos con influencias funcionales.

dart pub | dart compile

Dart vs Otros Lenguajes

JS **vs JavaScript/TypeScript:** Sintaxis similar (var, async/await, funciones flecha), pero con null safety de serie y mejor rendimiento.

vs Java: Tipado fuerte, clases e interfaces similares, pero con compilación nativa y código multiplataforma.

Historia: Dart nació en 2011, pero su popularidad explotó en 2017 con la llegada de Flutter.

Flujo de Compilación

Desarrollo

Dart → JIT → Ejecución rápida

Producción

Dart → AOT → Binario nativo

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Configuración del Entorno

Instalación del SDK

1. Descarga desde **dart.dev/get-dart**
2. Extrae y añade la carpeta bin al PATH
3. Verifica la instalación:

```
dart --version
```

Herramientas Incluidas

dart pub
Gestión de paquetes y dependencias

dart compile
Generación de ejecutables nativos

IDEs Recomendados

VS Code + Extensión Dart
Ligero y potente

Android Studio + Plugin
Ideal para Flutter

DartPad (Online)
Sin instalación: dartpad.dev

Crear Primer Proyecto CLI

```
dart create mi_proyecto
```

```
cd mi_proyecto
```

```
dart run
```

Estructura de Carpetas

```
mi_proyecto/
    ├─ bin/→ Código principal
    │  └─ mi_proyecto.dart
    ├─ lib/→ Librerías y módulos
    └─ pubspec.yaml→ Dependencias
```

Compilar a ejecutable: `dart compile exe main.dart`

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Sintaxis Básica de Dart

Declaración de Variables

```
var nombre = "Ana"; // Tipo inferido
dynamic dato = 42; // Puede cambiar
final fecha = DateTime.now(); // Asignación úni
const pi = 3.1416; // Compilación
```

Operadores Null-Aware

```
??→ Valor por defecto
?.→ Acceso seguro
! → Afirmación (cuidado)
```

Tipos Básicos

- **int:** Números enteros
- **double:** Decimales
- **bool:** true/false
- **String:** Texto
- **List:** Lista ordenada
- **Set:** Sin duplicados
- **Map:** Clave-valor
- **dynamic:** Cualquier tipo

Control de Flujo

If-Else

```
if (edad >= 18) { print("Mayor"); }
```

Switch con Pattern Matching

```
switch (dia) { case"Lunes"||"Martes": ... }
```

Bucles

```
for (var i = 0; i < 5; i++) { ... }
for (var fruta in frutas) { ... }
while (contador < 3) { ... }
```

Operador Cascade (..)

```
var buffer = StringBuffer()
  ..write("Hola ")
  ..write("Mundo");
```

Olga M. Moreno Martín

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Funciones y Parametrización Avanzada

Parámetros Opcionales

Se colocan entre corchetes y pueden tener valores por defecto.

```
void mostrar(String texto, [int veces = 1]) { ... }

// Uso:
mostrar("Hola"); // 1 vez
mostrar("Hola", 3); // 3 veces
```

Parámetros Nombrados

Se especifican por nombre, más legibles.

```
void crearUsuario({
  required String nombre,
  int edad = 18,
}) { ... }

// Uso:
        crearUsuario(nombre: "Ana");
```

Combinando Ambos

Parámetros posicionales y nombrados juntos.

```
void registrar(String id, {
  String? nombre,
  int edad = 0,
}) { ... }
```

Funciones Anónimas (Lambdas)

```
var sumar = (int a, int b) {
  return a + b;
};

print(sumar(3, 4)); // 7
```

Arrow Functions (=>)

Sintaxis reducida para funciones que retornan una expresión.

Uso común: Como argumentos de funciones de orden superior.

Lexical Scope

Las funciones internas acceden a variables de funciones externas.

Closures

```
Function contador() {
  int cuenta = 0;
  return () {
    cuenta++;
    print(cuenta);
  };
}

var c = contador();
c(); // 1
c(); // 2
```

Concepto clave: La función interna "recuerda" el estado externo.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Colecciones y Null Safety

List (Listas)

Ordenadas, permiten duplicados, indexadas desde 0.

```
var frutas = ["Manzana", "Pera"];
frutas.add("Uva");
print(frutas[0]); // Manzana
```

Métodos útiles:

map: Transforma

where: Filtra

fold: Reduce

Set (Conjuntos)

No permiten duplicados, ideales para valores únicos.

```
var numeros = {1, 2, 3, 3};
print(numeros); // {1, 2, 3}
numeros.add(4);
```

Map (Diccionarios)

Pares clave-valor, claves únicas.

```
var persona = {
  "nombre": "Ana",
  "edad": 30
};
print(persona["nombre"]); // Ana
```

Null Safety: Seguridad de Nulos Integrada

Dart utiliza null safety para evitar errores en tiempo de ejecución al acceder a variables que pueden ser nulas.

Tipos Anulables (?)

```
String? nombre;
nombre = null; // Válido
```

Operador ! (Afirmación)

```
String? texto = "Hola";
print(texto!.length); // 4
```

⚠️ Usar con cuidado

Operador ?. (Acceso Seguro)

```
String? saludo;
print(saludo?.length); // null
```

✅ No lanza error

Operador ?? (Valor por Defecto)

```
String? usuario;
print(usuario ?? "Invitado");
// "Invitado"
```

Ejemplo Combinado:

```
String? nombre;
print((nombre ?? "Desconocido").toUpperCase());
```

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Programación Orientada a Objetos en Dart

Clases y Constructores

```
class Persona {
  String nombre;
  int edad;
  Persona(this.nombre, this.edad);
  void saludar() => print("Hola, soy $nombre");
}
```

Constructores con Nombre

```
class Punto {
  double x, y;
  Punto(this.x, this.y);
  Punto.origen() : x = 0, y = 0;
}
var p1 = Punto(5, 3);
var p2 = Punto.origen();
```

Factory (Singleton)

```
class Conexion {
  static final Conexion _instancia = Conexion._interna();
  Factory Conexion() => _instancia;
  Conexion._interna();
}
```

Getters y Setters

Herencia (extends)

```
class Animal {
  void hacerSonido() => print("Sonido");
}
Class Perro extends Animal {
  @override
  void hacerSonido() => print("Guau");
}
```

Clases Abstractas

```
abstract class Figura {
  double area();
}
```

Mixins (with)

```
mixin Volador {
  void volar() => print("Volando");
}
Class Pajaro with Volador {}
```

Interfaces

```
class  Documento
  Implements Imprimible { ... }
```

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Características Avanzadas del Lenguaje

Extensions

Añaden métodos o propiedades a clases existentes sin modificarlas.

```
extension StringMayus onString {
  String primeraMayus() {
    if (isEmpty) return this;
    return this[0].toUpperCase() + substring(1);
  }
}

// Uso:
print("hola mundo".primeraMayus());
// Hola mundo
```

Ventaja: Añade utilidades a tipos nativos o de librerías externas.

Generators

Producen elementos bajo demanda (lazy evaluation).

sync* (Síncrono):

```
Iterable contarHasta(int max) sync* {
  for (int i = 1; i <= max; i++) {
    yield i; // Devuelve y pausa
  }
}
```

async* (Asíncrono):

```
Stream contarAsync(int max) async* { ... }
```

Usos: Lectura progresiva, streams de eventos, grandes listas.

Callable Classes

Permiten usar objetos como funciones.

```
classSuma {
  int call(int a, int b) => a + b;
}

var sumar = Suma();
print(sumar(3, 4)); // 7
```

Ventaja: Objetos que representan operaciones, más naturales de usar.

Ejemplo: Generador Síncrono

```
void main() {
  for (var n in contarHasta(5)) {
    print(n); // 1 2 3 4 5
  }
}
```

Concepto clave: yield devuelve un valor y "pausa" la función, manteniendo su estado para la próxima llamada.

Ejemplo: Generador Asíncrono

```
void main() async {
  awaitfor (var n in contarAsync(3)) {
    print(n); // 1 por segundo
  }
}
```

Olga M. Moreno Martín

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Asincronía y concurrencia

Futures

Representan un valor que estará disponible en el futuro.

```
Future obtenerDatos() async {
  awaitFuture.delayed(Duration(seconds: 2));
  return"Datos recibidos";
}
// Uso:
var datos = await obtenerDatos();
```

Manejo de errores:

try { ... } catch (e) { ... }

Streams

Secuencia de datos que llega con el tiempo.

```
Stream contadorStream(int max) async* {
  for (int i = 1; i <= max; i++) {
    awaitFuture.delayed(Duration(seconds: 1));
    yield i;
  }
}
```

Uso común:

Eventos de usuario, lectura de sensores, comunicación en tiempo real.

Isolates

Hilos independientes con memoria propia.

```
import'dart:isolate';
void tareaPesada(SendPort sendPort) {
  var resultado = 0;
  for (var i = 0; i < 100000000; i++) {
    resultado += i;
  }
  sendPort.send(resultado);
}
```

Ventaja:

Evita bloqueos en operaciones intensivas de CPU.

Stream con StreamController

```
import'dart:async';
void main() {
  final controller = StreamController<String>();
  controller.stream.listen((dato) {
    print("Recibido: $dato");
  });
  controller.add("Hola");
  controller.close();
}
```

Tip: Usa await para escribir código asíncrono como si fuera síncrono, mejorando la legibilidad.

Comparativa de Enfoques

Futures →	Un valor futuro. Único resultado.
Streams →	Múltiples valores en el tiempo.
Isolates →	Paralelismo real con memoria separada.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Buenas prácticas y documentación

Convenciones de Nombres

lowerCamelCase: variables y funciones

UpperCamelCase: clases y enums

lowerCamelCase: constantes

_privado: antepone _ para privacidad

Código Limpio

- Funciones cortas: una función = una cosa
- Evitar "números mágicos": usar constantes
- Comentarios claros: explicar el "por qué"
- Agrupar código relacionado

Organización de Archivos

```
lib/
  ├── models/ # Clases de datos
  ├── services/ # Lógica
  ├── utils/ # Funciones de ayuda
  └── main.dart
```

Documentación con Comentarios ///

```dart
/// Clase que representa un usuario en el sistema.
/// Contiene información básica como [nombre] y [edad].
class Usuario {
  /// Nombre completo del usuario.
  final String nombre;
  /// Edad del usuario en años.
  final int edad;
}
```

Genera documentación automática

Mejora la comprensión del código

Null Safety Correcto

❌ **Evitar:**

```dart
print(nombre!.length);
```

✅ **Preferir:**

```dart
print(nombre?.length ?? 0);
```

Documentación ///

Comentarios procesados por dart doc para generar documentación HTML.

Linting

Reglas automáticas para mantener estilo uniforme. Archivo: analysis_options.yaml

Principios SOLID

DRY, KISS, YAGNI para mantenibilidad.

TEMA · EXPLICACIÓN

PROMETEO by thePower

CONCEPTO

Pruebas en Dart

Configuración

Añade la dependencia en pubspec.yaml:

```
dev_dependencies:
  test: ^1.25.0
```

Estructura de Archivos

```
mi_proyecto/
  ├─ lib/calculadora.dart
  └─ test/calculadora_test.dart
```

Ejecución

```
dart test
```

Los archivos deben terminar en _test.dart

Ejemplo Completo de Test

```
import'package:test/test.dart';
import'../lib/calculadora.dart';

void main() {
  group('Pruebas calculadora', () {
    test('Suma correcta', () {
      expect(sumar(2, 3), equals(5));
    });
  });
}
```

setUp()

Se ejecuta antes de cada test.

tearDown()

Se ejecuta después de cada test.

Cobertura de Pruebas

```
dart test --coverage=coverage
```

PROMETEO
by thePower



¿Preguntas?

Espacio abierto para dudas, comentarios o reflexiones sobre lo visto hoy